

Node.js – Theory Assignment Answers

1. Introduction to Node.js

A. History and Evolution of Node.js

Node.js was created by Ryan Dahl and released in 2009. Dahl was motivated by the limitations of the Apache HTTP Server in handling concurrent connections. He built Node.js on Chrome's V8 JavaScript engine, enabling JavaScript to run outside the browser for the first time as a server-side technology.

Architecture: Node.js uses a single-threaded, event-driven, non-blocking I/O model. At its core is the **Event Loop**, which delegates I/O operations to the operating system and executes callbacks when results are ready. This makes it highly efficient for I/O-bound tasks without spawning new threads per request.

Key Features:

- Asynchronous and Non-Blocking I/O
- Single-threaded Event Loop architecture
- Fast execution via the V8 engine
- npm ecosystem — the world's largest package registry
- Cross-platform (Windows, macOS, Linux)
- Supports real-time applications (chat, live feeds)

B. Node.js vs Traditional Server-Side Technologies (PHP & Java)

The table below summarises the major differences:

Concurrency Model: Node.js – Event-driven, non-blocking single thread; PHP – Multi-threaded, blocking I/O; Java – Multi-threaded with thread pools (JVM).

Language: Node.js – JavaScript (full-stack sharing); PHP – PHP (server-only); Java – Java (verbose, strongly typed).

Performance: Node.js – Excellent for I/O-heavy tasks; PHP – Moderate; slower for high concurrency; Java – High for CPU-heavy tasks with JVM JIT.

Package Manager: Node.js – npm / yarn; PHP – Composer; Java – Maven / Gradle.

Use Case: Node.js – APIs, real-time apps, microservices; PHP – CMS, traditional web pages; Java – Enterprise-grade large-scale systems.

Learning Curve: Node.js – Low–Medium (JS knowledge reused); PHP – Low; Java – High.

Node.js shines in scenarios requiring high concurrency with many simultaneous connections (e.g., chat applications, REST APIs). PHP is easier to deploy for traditional web pages, while Java is preferred for large enterprise systems requiring strong typing and extensive frameworks.

2. Node.js Environment Setup – Role of npm

npm (Node Package Manager) is the default package manager for Node.js. It serves three main purposes:

- **Package Installation:** Downloads and installs open-source libraries (packages) from the npm registry into a project.
- **Dependency Management:** Tracks all packages a project depends on via package.json and package-lock.json, ensuring reproducible builds.
- **Script Running:** Allows developers to define and run custom scripts (e.g., start, test, build) through the 'scripts' field in package.json.

Common npm Commands:

- **npm init / npm init -y** – Initialises a new Node.js project and creates package.json.
 - **npm install <package>** – Installs a package and adds it as a dependency.
 - **npm install --save-dev <package>** – Installs a package as a development dependency.
 - **npm install** – Installs all dependencies listed in package.json.
 - **npm uninstall <package>** – Removes a package from the project.
 - **npm update** – Updates all packages to their latest compatible versions.
 - **npm run <script>** – Executes a script defined in package.json.
 - **npm list** – Displays the installed packages tree.
 - **npm publish** – Publishes a package to the npm registry.
 - **npm audit** – Checks for known security vulnerabilities in dependencies.
-

3. Module System in JavaScript

A module system allows developers to split code into separate, reusable files. This improves maintainability, readability, and separation of concerns. JavaScript has two primary module systems:

A. CommonJS (CJS)

CommonJS is the original module system used in Node.js. Modules are loaded **synchronously** at runtime. Each file is its own module with its own scope.

- Uses **require()** to import modules.
- Uses **module.exports** or **exports** to expose functionality.
- Loaded synchronously — suitable for server-side where files are on disk.
- Dynamic imports possible (require inside conditions/functions).

B. ES Modules (ESM)

ES Modules are the official ECMAScript standard, introduced in ES6 (2015). They are now supported natively in modern browsers and Node.js (v12+).

- Uses **import** to bring in modules.
- Uses **export** / **export default** to expose functionality.
- Loaded **asynchronously** — better suited for browsers.
- Static analysis: imports are resolved at compile time, enabling tree shaking.
- Strict mode is enabled by default.

Significance: Both systems encourage modular code organisation, reduce global namespace pollution, and make code easier to test and reuse. ESM is the future standard, while CJS remains dominant in existing Node.js codebases.

4. JavaScript APIs and the Fetch API

An **API (Application Programming Interface)** is a set of rules and protocols that allows one software application to communicate with another. In the context of the web, APIs expose data and functionality over HTTP, typically returning JSON or XML.

JavaScript can interact with APIs using several mechanisms:

- **XMLHttpRequest (XHR):** The original way to make HTTP requests; verbose and callback-based.
- **Fetch API:** A modern, promise-based interface for making HTTP requests.
- **Axios:** A popular third-party library that wraps Fetch/XHR with extra features.

The Fetch API

The Fetch API is a built-in browser API (also available in Node.js v18+) that provides a clean interface for making network requests. It is promise-based, making it compatible with `async/await` syntax.

- Returns a **Promise** that resolves to a **Response** object.
 - Supports all HTTP methods: GET, POST, PUT, DELETE, PATCH.
 - Allows setting custom headers, body, and request mode (e.g., cors, no-cors).
 - The Response object provides methods like `.json()`, `.text()`, `.blob()` for parsing.
 - Does not reject on HTTP error status codes (e.g., 404, 500) — must be checked manually via `response.ok`.
-

5. Functional Programming Concepts in JavaScript

Functional Programming (FP) is a programming paradigm that treats computation as the evaluation of mathematical functions and avoids changing state or mutable data.

Core Principles:

- **Pure Functions:** A function is pure if it always returns the same output for the same input and has no side effects (does not modify external state). Pure functions are predictable and easy to test.

- **Immutability:** Data is never modified after creation. Instead of changing an object, you create a new one. This prevents unexpected bugs caused by shared mutable state.
- **First-Class Functions:** Functions are treated as values — they can be stored in variables, passed as arguments, or returned from other functions.
- **Higher-Order Functions:** Functions that accept other functions as arguments or return functions. Examples: `map()`, `filter()`, `reduce()`.
- **Function Composition:** Combining simple functions to build more complex ones, improving code reuse and readability.
- **Declarative Style:** FP focuses on what to do rather than how to do it, leading to more concise and readable code.

Key Array Methods in JavaScript FP:

- **`map()`:** Transforms each element of an array and returns a new array.
 - **`filter()`:** Returns a new array containing only elements that satisfy a condition.
 - **`reduce()`:** Accumulates array elements into a single value using a reducer function.
-

6. Understanding Node.js Modules – CommonJS vs ES Modules

Node.js was built around the **CommonJS (CJS)** module system. As JavaScript evolved, **ES Modules (ESM)** became the standard. Node.js now supports both.

CommonJS in Node.js:

- Every file is automatically wrapped in a module wrapper function, giving it its own scope.
- `require()` loads modules synchronously.
- `module.exports` defines what the module exposes.
- The `exports` object is a shorthand reference to `module.exports`.
- All built-in Node.js modules (`fs`, `http`, `path`) use CommonJS.

How CommonJS Differs from ES Modules:

- **Syntax:** CJS – `require()` / `module.exports` | ESM – `import` / `export`
 - **Loading:** CJS – Synchronous | ESM – Asynchronous
 - **Scope:** CJS – Module scoped by wrapper | ESM – Strict mode, own scope
 - **Dynamic Imports:** CJS – Yes (require inside functions) | ESM – Limited (`import()` function)
 - **File Extension:** CJS – `.js` (default) | ESM – `.mjs` or `package.json` type: `module`
 - **Tree Shaking:** CJS – Not supported | ESM – Supported (static analysis)
 - **this at top level:** CJS – `module.exports` | ESM – undefined
-

7. File System Module in Node.js

The **fs (File System)** module is a core Node.js module that provides an API for interacting with the file system. It allows reading, writing, updating, deleting, and watching files and directories, making it essential for building file-based applications, log systems, data pipelines, and more.

Importance of the fs Module:

- Enables persistent data storage without a database.
- Used for reading configuration files (e.g., JSON config).
- Essential for logging, report generation, and file upload/download handling.
- Provides both synchronous and asynchronous APIs.

Synchronous vs Asynchronous File Operations

Synchronous operations (e.g., `fs.readFileSync`, `fs.writeFileSync`) block the execution of the program until the operation completes. They are simpler to write but inappropriate for high-concurrency production servers because they freeze the event loop.

Asynchronous operations (e.g., `fs.readFile`, `fs.writeFile`) use callbacks, promises, or `async/await`. They do not block the event loop — Node.js can continue handling other requests while the I/O operation runs in the background. This is the recommended approach for server-side code.

- **Blocking:** Sync – Yes — halts all other operations | Async – No — event loop continues
 - **Use case:** Sync – Scripts, CLI tools, startup config loading | Async – Web servers, APIs, production apps
 - **Error handling:** Sync – `try/catch` block | Async – Callback error param or `.catch()`
 - **Performance:** Sync – Poor under concurrency | Async – Excellent under concurrency
-

8. HTTP and Web Servers in Node.js

Node.js provides a built-in **http** module that allows developers to create HTTP servers and clients without any third-party libraries.

The Request-Response Lifecycle

1. Client sends an HTTP request (method, URL, headers, optional body) to the server.
2. Node.js's `http` module receives the request and emits a 'request' event.
3. The event loop picks up the event and passes it to the registered callback (`req`, `res`).
4. The callback processes the request — reads parameters, queries a database, reads a file, etc.
5. The server constructs a response with a status code, headers, and body.
6. `res.end()` or `res.write()` sends the response back to the client.
7. The connection is closed (or kept alive for HTTP/1.1 keep-alive).

How Node.js Handles HTTP Requests: Node.js uses its non-blocking I/O and event loop to handle thousands of concurrent connections on a single thread. When a request arrives, it is queued in the event

loop. I/O operations (like DB queries) are delegated to libuv's thread pool, and when complete, the callback is pushed back to the event loop for processing.

9. Express.js Framework – Advantages over the http Module

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications. It abstracts away the complexities of the built-in http module.

Advantages of Express over the built-in http module:

- **Simplified Routing:** Express provides a clean, declarative routing API (`app.get()`, `app.post()`, `app.put()`, etc.) instead of manually parsing `req.url` and `req.method`.
 - **Middleware Support:** Express has a powerful middleware pipeline. Functions can be inserted into the request-response cycle to handle logging, authentication, body parsing, CORS, etc.
 - **Request/Response Enhancements:** Express extends the native `req` and `res` objects with useful methods like `res.json()`, `res.send()`, `res.status()`, `req.params`, `req.query`, `req.body`.
 - **Template Engine Integration:** Easily integrates with template engines (EJS, Pug, Handlebars) for server-side HTML rendering.
 - **Static File Serving:** `express.static()` middleware serves static files with one line of code.
 - **Error Handling:** Provides a standardised error-handling middleware signature (`err`, `req`, `res`, `next`).
 - **Large Ecosystem:** Thousands of compatible middleware packages available on npm.
 - **Router Module:** `express.Router()` allows organising routes into modular, mountable route handlers.
-

10. RESTful API Principles

REST (Representational State Transfer) is an architectural style for designing networked applications. A RESTful API follows REST constraints to enable scalable, stateless client-server communication over HTTP.

Core REST Principles:

- **Client-Server Architecture:** The client and server are independent. The client handles the UI; the server handles data storage and business logic. They communicate only through a well-defined interface.
- **Statelessness:** Each request from the client must contain all the information the server needs to process it. The server does not store client session state between requests.
- **Cacheability:** Responses must define themselves as cacheable or non-cacheable. Proper caching improves performance and scalability.
- **Uniform Interface:** A consistent interface between components, defined by resource identification (URLs), resource manipulation through representations, self-descriptive messages, and HATEOAS.

- **Layered System:** The client cannot tell whether it is connected directly to the end server or an intermediary (load balancer, cache, gateway).
- **Code on Demand (optional):** Servers can extend client functionality by sending executable code (e.g., JavaScript).

HTTP Methods in REST:

- **GET:** Retrieve a resource — safe and idempotent.
- **POST:** Create a new resource — not idempotent.
- **PUT:** Replace/update an existing resource — idempotent.
- **PATCH:** Partially update a resource.
- **DELETE:** Remove a resource — idempotent.

Importance in Web Development: RESTful APIs provide a standardised, language-agnostic way for frontend applications, mobile apps, and third-party services to communicate with a backend. They are scalable, easy to understand, and supported by every HTTP client.

11. Asynchronous Programming in Node.js

Asynchronous programming is fundamental to Node.js. Because Node.js runs on a single thread, blocking operations (like file I/O or database queries) would freeze the entire application. Async programming allows Node.js to initiate an operation and continue executing other code while waiting for the result.

A. Callbacks

A **callback** is a function passed as an argument to another function, to be executed once an asynchronous operation completes. It is the original pattern used in Node.js.

Drawback: Deeply nested callbacks create 'Callback Hell' or the 'Pyramid of Doom' — code that is hard to read, maintain, and debug.

B. Promises

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation. It can be in one of three states:

- **Pending:** Initial state — neither fulfilled nor rejected.
- **Fulfilled:** The operation completed successfully.
- **Rejected:** The operation failed.

Promises support chaining via `.then()` and error handling via `.catch()`, making code flatter and more readable than nested callbacks. `Promise.all()` runs multiple promises in parallel; `Promise.race()` resolves with the first settled promise.

C. Async/Await

async/await is syntactic sugar built on top of Promises, introduced in ES2017. An async function always returns a Promise. The await keyword pauses execution of the async function until the awaited Promise settles, making asynchronous code look and behave like synchronous code.

- Eliminates .then() chaining — code is linear and readable.
 - Error handling uses familiar try/catch blocks.
 - Easier to debug (stack traces are cleaner).
 - Works with any Promise-based API.
-

12. Database Integration – SQL vs NoSQL

Databases are essential for persisting application data. Node.js can connect to both SQL (relational) and NoSQL (non-relational) databases.

SQL Databases

SQL databases store data in structured tables with predefined schemas. They use Structured Query Language (SQL) for querying and support ACID transactions.

- Examples: MySQL, PostgreSQL, SQLite, Microsoft SQL Server.
- Best for: Complex queries, relationships, financial data, structured data.
- Node.js libraries: mysql2, pg (node-postgres), Sequelize (ORM), Knex.js.

NoSQL Databases

NoSQL databases store data in flexible formats: documents (JSON), key-value pairs, graphs, or wide columns. They are schema-less and horizontally scalable.

- Examples: MongoDB (document), Redis (key-value), Cassandra (wide-column), Neo4j (graph).
- Best for: Unstructured data, high scalability, real-time apps, flexible schemas.
- Node.js libraries: mongoose (MongoDB ODM), ioredis (Redis), mongodb (native driver).

Key Comparison:

- **Schema:** SQL – Fixed, predefined | NoSQL – Dynamic, flexible
- **Scalability:** SQL – Vertical (scale up) | NoSQL – Horizontal (scale out)
- **Transactions:** SQL – Full ACID support | NoSQL – Eventual consistency (varies)
- **Query Language:** SQL – SQL | NoSQL – API-specific (e.g., MongoDB Query Language)
- **Relationships:** SQL – Joins across tables | NoSQL – Embedded documents or references

Connecting Node.js to Databases: Node.js connects to databases using driver libraries or ORMs/ODMs. For MongoDB, Mongoose is the most popular ODM. It allows defining schemas, models, and provides validation, middleware hooks, and query building.

13. Error Handling in Node.js

Proper error handling is critical in Node.js applications to ensure stability, provide meaningful feedback to users, and facilitate debugging.

Importance of Error Handling:

- Prevents application crashes from unhandled exceptions.
- Improves user experience with meaningful error messages.
- Aids debugging and monitoring in production.
- Ensures resources (DB connections, file handles) are properly released.
- Maintains data integrity by rolling back failed operations.

Error Handling Techniques in Node.js:

- **try/catch:** Used with synchronous code and `async/await`. Wraps potentially failing code; the catch block handles thrown errors.
 - **Callback Error-First Pattern:** Node.js convention: callbacks receive `(err, result)`. Always check if `err` is truthy before processing results.
 - **Promise .catch():** Handles rejected promises in promise chains. Prevents `UnhandledPromiseRejection` warnings.
 - **async/await + try/catch:** Clean way to handle async errors. The catch block catches both synchronous throws and rejected promises.
 - **EventEmitter 'error' event:** Streams and other EventEmitters emit an 'error' event. If unhandled, it throws an exception. Always attach an error listener.
 - **Express Error Middleware:** A special 4-argument middleware (`err, req, res, next`) that centrally handles all errors passed via `next(err)`.
 - **process.on('uncaughtException'):** Catches synchronous exceptions that weren't handled. Should only be used for cleanup — the process should exit afterward.
 - **process.on('unhandledRejection'):** Catches unhandled promise rejections globally. Essential for catching missed `.catch()` handlers.
-

14. Testing and Debugging in Node.js

Importance of Testing: Testing is the process of verifying that software behaves as expected. It is a cornerstone of software quality that reduces bugs, improves maintainability, and provides confidence when refactoring or adding features.

- Catches bugs early in the development cycle when they are cheapest to fix.
- Acts as living documentation describing how functions/routes should behave.
- Enables safe refactoring — tests catch regressions.
- Essential for CI/CD pipelines — automated tests gate deployments.

- Improves code design — testable code is usually better structured.

Popular Testing Frameworks for Node.js:

- **Mocha:** A flexible test runner that supports async tests, before/after hooks, and multiple reporters. Commonly paired with Chai for assertions.
- **Chai:** A BDD/TDD assertion library. Provides three assertion styles: should, expect, and assert.
- **Jest:** An all-in-one testing framework by Meta. Includes test runner, assertion library, mocking, and coverage reporting. Zero configuration needed.
- **Supertest:** HTTP assertion library for testing Express routes. Makes requests to your app without starting a real server.
- **Sinon.js:** Provides test spies, stubs, and mocks for isolating units of code.
- **Jasmine:** A behaviour-driven development framework with built-in assertions and a clean syntax.
- **AVA:** A concurrent test runner that runs tests in parallel, making test suites faster.

Types of Testing: Unit Testing (individual functions), Integration Testing (multiple components together), End-to-End Testing (full user workflows), and Load Testing (performance under stress).

15. Middleware in Express.js

Middleware in Express.js refers to functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle. Middleware functions can:

- Execute any code.
- Make changes to the request and response objects.
- End the request-response cycle.
- Call the next middleware in the stack via next().

Role in Request Handling: Middleware forms a pipeline. Each incoming request passes through middleware functions in the order they are registered. This pipeline approach separates cross-cutting concerns (logging, authentication, parsing) from route logic.

Types of Middleware in Express:

- **Application-level:** Bound to the app instance using app.use() or app.METHOD(). Applied to all or specific routes.
 - **Router-level:** Bound to an express.Router() instance. Scoped to specific route groups.
 - **Error-handling:** Has four arguments (err, req, res, next). Must be defined after all routes.
 - **Built-in:** express.json() (parses JSON bodies), express.urlencoded() (parses URL-encoded bodies), express.static() (serves static files).
 - **Third-party:** e.g., cors, helmet, morgan, compression, express-rate-limit.
-

16. Authentication and Security in Web Applications

Importance of Authentication and Security: Authentication verifies the identity of users, while authorisation determines what they can access. Security is paramount because web applications handle sensitive personal, financial, and business data.

- Prevents unauthorised access to sensitive data and features.
- Protects against common attacks: XSS, CSRF, SQL injection, brute force.
- Ensures compliance with regulations (GDPR, HIPAA, PCI-DSS).
- Builds user trust and protects the reputation of the application.

Common Authentication Strategies in Node.js:

- **Session-Based Authentication:** The server creates a session after login, stores it server-side, and sends a session ID to the client via a cookie. The cookie is sent with every request for verification. Stateful — requires session storage (memory, Redis, DB).
- **JSON Web Tokens (JWT):** After successful login, the server signs a token containing user claims and sends it to the client. The client stores it (localStorage or cookie) and sends it in the Authorization header. The server verifies the token's signature on each request. Stateless — no server-side session storage needed. Best for APIs and microservices.
- **Passport.js:** A popular Node.js authentication middleware that supports 500+ strategies including Local (username/password), OAuth (Google, Facebook, GitHub), OpenID, and JWT. It abstracts authentication logic and integrates cleanly with Express.
- **OAuth 2.0 / OpenID Connect:** Delegates authentication to trusted providers (Google, GitHub). The user logs in with their existing account; the provider returns an access token. Used for 'Login with Google' type flows.
- **Basic Authentication:** Credentials (username:password) are Base64-encoded and sent in the Authorization header. Simple but insecure over plain HTTP — must always use HTTPS.
- **API Keys:** A unique key issued to clients for accessing an API. Simple but does not represent a user — better for machine-to-machine communication.

Security Best Practices in Node.js:

- Always hash passwords using bcrypt or Argon2 — never store plain text.
 - Use HTTPS in production to encrypt data in transit.
 - Validate and sanitise all user input to prevent injection attacks.
 - Set security HTTP headers using the helmet middleware.
 - Implement rate limiting to prevent brute-force attacks.
 - Use environment variables for secrets — never hardcode credentials.
 - Keep dependencies updated and audit with npm audit.
 - Implement CORS correctly to control which origins can access your API.
-

